REVISION CHEAT SHEET:

SQL Server Concurrency, Locking & Isolation Levels

Complete notes - locks - blocking - deadlocks - isolation - row versioning - practical labs

INDEX - Main Topics & Important Subtopics

Study flow: concurrency -> locks -> blocking/deadlocks -> optimistic/pessimistic control -> isolation anomalies -> isolation levels -> RCSI/SNAPSHOT -> practical troubleshooting.

- Concurrency Fundamentals
- Locking and Lock Granularity
- Shared, Exclusive, Update and Intent Locks
- Lock Compatibility
- Blocking and Head Blockers
- Deadlocks and Prevention
- Lock Escalation
- Pessimistic vs Optimistic Concurrency
- Isolation and Read Phenomena
- READ UNCOMMITTED and READ COMMITTED
- REPEATABLE READ and SERIALIZABLE
- SNAPSHOT Isolation
- READ COMMITTED SNAPSHOT - RCSI
- Isolation Comparison and Selection
- Practical Fundraising Scenario
- RCSI / SNAPSHOT Settings
- Concurrency Mistakes and Best Practices
- Practical Blocking and Deadlock Labs
- Interview Questions
- Final Revision and Memory Rules

Concurrency means:

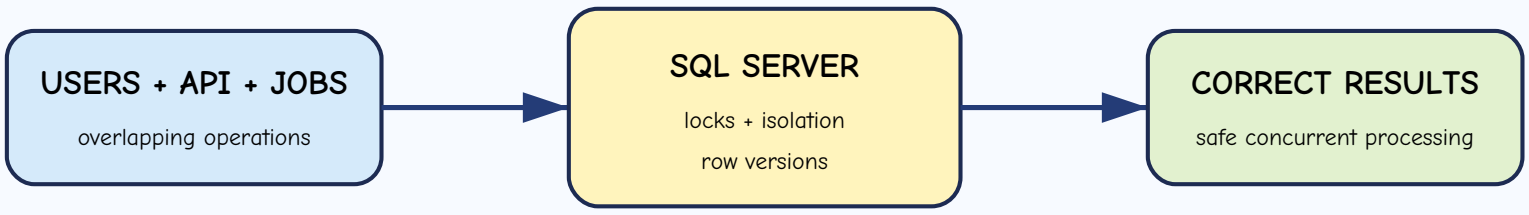
Multiple users or processes accessing the database at the same time.

A real application is rarely used by only one person.

In your `FundraisingDb`, at the same moment:

AT A GLANCE

Concurrency - Simultaneous Work with Controlled Access



EXAMPLE / FLOW

```
Fundraiser Sarah
→ reads John Smith's Constituent record

API request 2
→ inserts John's new Gift

Background process
→ creates GiftAllocation

Another fundraiser
→ updates a Task

Reporting query
→ calculates Campaign totals
```

SQL Server must allow all of these operations to happen concurrently while still protecting the correctness of the data.

That is why SQL Server needs:

EXAMPLE / FLOW

```
Transactions
Locks
Isolation levels
Blocking control
Deadlock detection
Row versioning
```

The basic idea is:

EXAMPLE / FLOW

```
Many users
     |
     v
Same database
     |
     v
SQL Server controls access
     |
     +---- Locks
     |
     +---- Row versions
     |
     +---- Isolation levels
     |
     v
Correct concurrent processing
```

What Is Concurrency?

Simple Definition

Concurrency means two or more database operations are happening during overlapping periods of time.

For example:

EXAMPLE / FLOW

```
User A
→ reads Gift 10000

User B
→ updates Gift 10000

User C
→ reads Campaign totals
```

These operations may happen at almost the same time.

Why Is Concurrency Important?

Without concurrency:

EXAMPLE / FLOW

```
User A works
↓
Everyone else waits
↓
User A finishes
↓
User B works
↓
Everyone else waits
```

That would make a web application extremely slow.

Instead SQL Server tries to allow:

EXAMPLE / FLOW

```
User A ──────────────────>

User B       ──────────────────>

User C    ──────────────────────>
```

to work simultaneously whenever their operations do not conflict.

The Concurrency Problem

Concurrency creates a problem:

> ! What happens when two users access the **same data** at the same time?

Example:

EXAMPLE / FLOW

```
Gift 10000
TotalAmount = $1,000
```

User A reads:

EXAMPLE / FLOW

```
$1,000
```

At the same moment User B changes it to:

EXAMPLE / FLOW

```
$1,200
```

SQL Server must decide:

EXAMPLE / FLOW

```
Can User A continue reading?

Should User B wait?

Should User A see old value?

Should User A see new value?

Should one transaction fail?
```

Those decisions are controlled mainly by:

EXAMPLE / FLOW

```
Locks
+
Isolation level
+
Row versioning
```

What Is Locking?

Simple Definition

A **lock** is a mechanism SQL Server uses to protect a resource while a transaction is reading or changing it.

Think of a lock like a sign saying:

EXAMPLE / FLOW

```
"I am currently using this resource.

Other transactions may or may not
be allowed to use it depending on
the type of lock I hold."
```

SQL Server uses locks to protect transaction integrity.

Simple Fundraising Example

Suppose:

EXAMPLE / FLOW

```
GiftId = 10000
TotalAmount = $1,000
```

Transaction A executes:

SQL

```
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;
```

SQL Server must protect that changed row.

Conceptually:

EXAMPLE / FLOW

```
Gift 10000
    |
    v
Exclusive Lock
    |
    v
Other conflicting changes must wait
```

Why Are Locks Needed?

Locks help prevent problems such as:

EXAMPLE / FLOW

```
Two transactions changing the same row incorrectly

A reader seeing partially changed data

Conflicting updates

Incorrect transaction results
```

Locks Are Usually Automatic

Normally you do not write:

EXAMPLE / FLOW

```
Take shared lock
Take exclusive lock
Release lock
```

SQL Server decides the required locks automatically based on:

EXAMPLE / FLOW

```
The SQL statement

The operation type

The isolation level

The number of affected rows

The execution plan
```

Lock Granularity

SQL Server can lock resources at different levels.

Important examples are:

EXAMPLE / FLOW

```
Key / Row

Page

Table

Database-related resources
```

Conceptually:

EXAMPLE / FLOW

```
Database
   |
   v
Table
   |
   v
Page
   |
   v
Row / Index Key
```

Row/Key Lock

Very fine-grained.

Example:

EXAMPLE / FLOW

```
Lock only Gift 10000
```

Advantages:

EXAMPLE / FLOW

```
More concurrency
Other Gift rows may remain accessible
```

Disadvantage:

EXAMPLE / FLOW

```
Thousands of row locks consume more lock-management resources
```

Page Lock

One SQL Server data page usually contains multiple rows.

Conceptually:

EXAMPLE / FLOW

```
Page
+-------------------+
| Gift 10000        |
| Gift 10001        |
| Gift 10002        |
| Gift 10003        |
+-------------------+
```

A page lock can protect several rows at once.

Table Lock

Protects the entire table.

Example:

EXAMPLE / FLOW

```
fundraising.Gift
      |
      v
TABLE LOCK
```

This requires fewer lock structures but reduces concurrency.

Important Rule

Do not assume:

EXAMPLE / FLOW

```
Row locks = always good

Table locks = always bad
```

For a query modifying millions of rows, thousands of tiny locks may be more expensive than a broader lock.

SQL Server chooses lock granularity dynamically.

Shared Lock - S Lock

Simple Definition

A **Shared Lock (S)** protects data being read.

Think:

EXAMPLE / FLOW

```
"I am reading this data.

Other readers may also read it,
but conflicting writers may need to wait."
```

Example

Transaction A:

SQL

```
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE GiftId = 10000;
```

Under normal lock-based `READ COMMITTED`, SQL Server may acquire a:

EXAMPLE / FLOW

```
Shared lock
```

while reading the row. `READ COMMITTED` prevents dirty reads; when `READ_COMMITTED_SNAPSHOT` is OFF, SQL Server uses shared locks for reads.

Shared Lock Compatibility

Two readers can normally coexist.

EXAMPLE / FLOW

```
Transaction A
Shared Lock
      |
      v
Gift 10000
      ^
      |
Transaction B
Shared Lock
```

This is allowed because:

EXAMPLE / FLOW

```
Reader + Reader
→ generally compatible
```

But Writer May Conflict

EXAMPLE / FLOW

```
Transaction A
S Lock
   |
   v
Gift 10000
   ^
   |
Transaction B
wants X Lock
```

Transaction B may have to wait until the conflicting shared lock is released.

Easy Memory

EXAMPLE / FLOW

```
S = Shared
S = SELECT/read protection
```

Exclusive Lock - X Lock

Simple Definition

An **Exclusive Lock (X)** protects data being modified.

Operations such as:

EXAMPLE / FLOW

```
INSERT
UPDATE
DELETE
```

require exclusive-style protection on changed data.

Example

SQL

```
UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;
```

SQL Server acquires an:

EXAMPLE / FLOW

```
X lock
```

on the modified resource.

Exclusive Means Exclusive

While Transaction A holds an X lock:

EXAMPLE / FLOW

```
Transaction A
X Lock
   |
   v
Gift 10000
```

another transaction cannot obtain another incompatible:

EXAMPLE / FLOW

```
X lock
```

on that same resource.

Important Rule

Data modifications take exclusive locks regardless of the transaction isolation level; those locks protect write consistency.

X Lock Lifetime

Traditionally, exclusive locks protecting modified data are retained until the transaction completes.

Example:

```sql
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;

-- transaction still open
```

A conflicting transaction may remain blocked until:

```sql
COMMIT;
```

or:

```sql
ROLLBACK;
```

Modern SQL Server can also use **optimized locking** when enabled, which can reduce how many row/page locks must be held for the full transaction, but the basic concurrency rule remains: uncommitted conflicting changes are protected.

Easy Memory

```
X = Exclusive
X = data modification
```

Update Lock - U Lock

Simple Definition

An **Update Lock (U)** is used when SQL Server reads a row that may later be updated.

It helps prevent a common deadlock pattern involving two sessions that both read the same row and then both try to convert their shared locks to exclusive locks.

Microsoft describes the Update lock as a lock used on resources that may later be updated and specifically notes its use in preventing a common type of deadlock.

Why Do We Need Update Locks?

Imagine:

EXAMPLE / FLOW

```
Transaction A
reads Gift 10000

Transaction B
reads Gift 10000
```

Both obtain:

EXAMPLE / FLOW

```
Shared locks
```

Now both decide:

EXAMPLE / FLOW

```
I want to update this Gift.
```

They both need:

EXAMPLE / FLOW

```
X lock
```

But each transaction may be holding a shared lock that blocks the other's conversion.

Update locks reduce this conversion-deadlock risk.

Conceptual Flow

EXAMPLE / FLOW

```
Read row for possible UPDATE
          |
          v
      U Lock
          |
          v
Actually update?
      /       \
    No         Yes
               |
               v
           Convert to X
```

Example with UPDLOCK

You may explicitly request update-style locking:

```sql
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift WITH (UPDLOCK)
WHERE GiftId = 10000;
```

This is an advanced locking pattern.

Do not add **UPDLOCK** to every query.

Use it only when a transaction genuinely follows:

```
read
→ decide
→ update same rows
```

and the locking behavior is intentionally part of the design.

Intent Locks

Intent locks confuse many beginners, so start with the purpose.

Simple Definition

An **intent lock** says:

> ! "I already have, or intend to obtain, a more detailed lock somewhere below this resource."

They help SQL Server manage hierarchical locking efficiently.

Lock Hierarchy

Think:

```
Table
  |
  v
Page
  |
  v
Row
```

Suppose SQL Server has an X lock on one row.

Instead of another transaction checking:

EXAMPLE / FLOW

```
every single row lock
```

to determine whether the table can be locked, SQL Server can look at the table-level intent lock.

Intent Shared - IS

Meaning:

EXAMPLE / FLOW

```
There are Shared locks,
or shared-style locks are intended,
at lower levels.
```

Example:

EXAMPLE / FLOW

```
Table: IS
   |
   v
Row 10000: S
```

Intent Exclusive - IX

Meaning:

EXAMPLE / FLOW

```
There are Exclusive locks,
or exclusive-style locks are intended,
at lower levels.
```

Example:

EXAMPLE / FLOW

```
Table: IX
   |
   v
Row 10000: X
```

Intent Update - IU

Indicates intent to obtain Update locks on lower-level resources.

SQL Server documents **IS**, **IU**, and **IX** as intent locks indicating shared, update, or exclusive locking intentions on subordinate resources.

Easy Intent-Lock Diagram

EXAMPLE / FLOW

```
fundraising.Gift table
        |
        | IX
        v
Data Page
        |
        | IX
        v
Gift 10000
        |
        | X
        v
updated row
```

The intent locks do not mean:

EXAMPLE / FLOW

```
whole table is exclusively locked
```

They mean:

EXAMPLE / FLOW

```
lower-level exclusive locks exist/intended.
```

Intent Locks Are Usually Automatic

You normally do not manually control:

EXAMPLE / FLOW

```
IS
IX
IU
```

SQL Server manages them.

For learning:

EXAMPLE / FLOW

```
IS
→ shared below

IX
→ exclusive below
```

```
IU
→ update below
```

is sufficient.

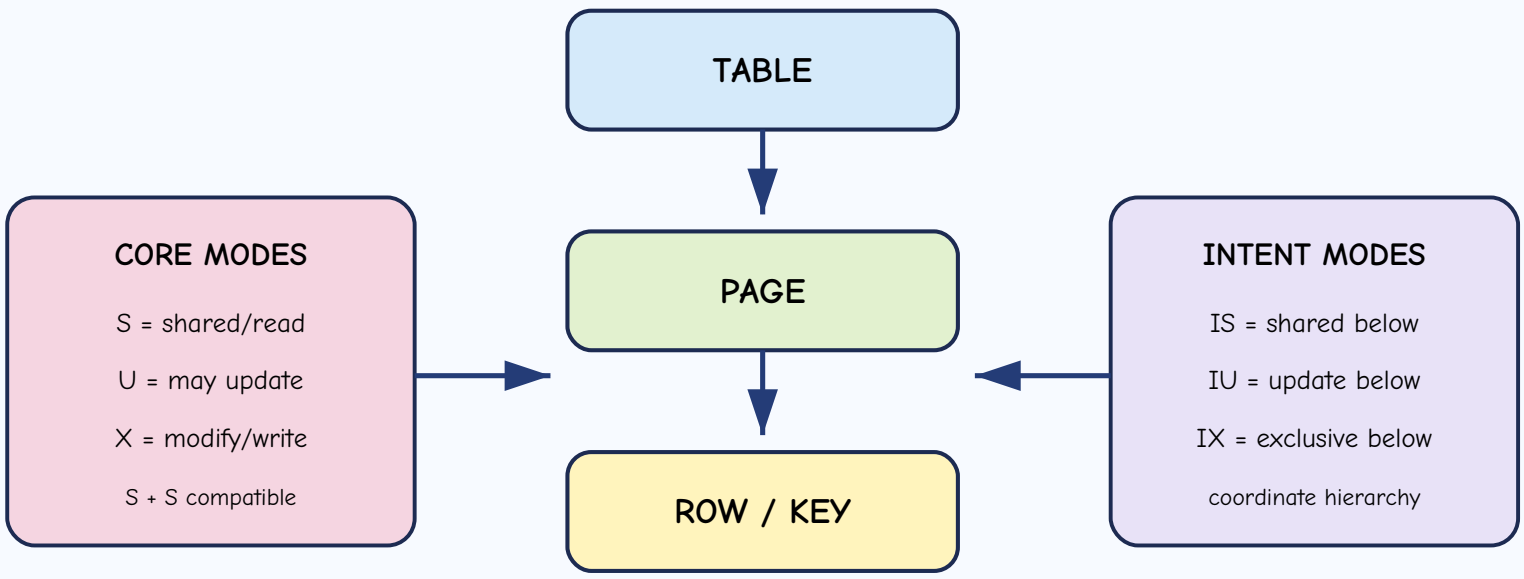
Basic Lock Compatibility

You do not need to memorize every SQL Server lock mode initially.

The most important relationships are:

LOCK MAP

Lock Hierarchy and Core Lock Modes



Locks can exist at different resource levels. Intent locks summarize lower-level locking so SQL Server can coordinate hierarchy checks efficiently.

Existing Lock	Requested S	Requested X
S	Usually compatible	Not compatible
X	Not compatible	Not compatible

Memory:

EXAMPLE / FLOW

```
S + S
→ Yes

S + X
→ No
```

```
X + S
→ No

X + X
→ No
```

Intent-lock compatibility is more detailed, but the purpose is mainly to coordinate hierarchical locks.

Blocking

Simple Definition

Blocking happens when one transaction must wait because another transaction holds an incompatible lock on a needed resource.

Blocking is not automatically an error.

Some blocking is a normal part of maintaining correct concurrent behavior.

Simple Blocking Example

Session 1:

SQL

```sql
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;

-- Do not COMMIT yet
```

Session 2:

SQL

```sql
UPDATE fundraising.Gift
SET TotalAmount = 1500
WHERE GiftId = 10000;
```

Session 2 may wait.

Diagram:

EXAMPLE / FLOW

```
Session 1
X Lock on Gift 10000
      |
      v
   Gift row
      ^
      |
```

```
Session 2
Needs X Lock
      |
      v
    WAITS
```

Blocking Ends When

Session 1 executes:

```sql
COMMIT;
```

or:

```sql
ROLLBACK;
```

Then the incompatible lock can be released and Session 2 can continue.

Normal Blocking vs Bad Blocking

Short blocking:

```
EXAMPLE / FLOW
10 ms
30 ms
100 ms
```

may be completely normal.

Problematic blocking:

```
EXAMPLE / FLOW
10 seconds
30 seconds
several minutes
```

may indicate:

```
EXAMPLE / FLOW
Long transactions

Slow queries

Poor indexes
```

```
Too many rows being modified

High isolation levels

Application forgetting to commit

User interaction inside transaction
```

Head Blocker

Suppose:

EXAMPLE / FLOW

```
Session 10 blocks Session 20

Session 20 blocks Session 30

Session 30 blocks Session 40
```

Flow:

EXAMPLE / FLOW

```
Session 10
   |
   v
Session 20
   |
   v
Session 30
   |
   v
Session 40
```

The important session is:

EXAMPLE / FLOW

```
Session 10
```

because it is the:

EXAMPLE / FLOW

```
head blocker
```

Microsoft recommends identifying the head blocker first when troubleshooting prolonged blocking.

Find Blocking Session

A useful DMV:

SQL

```
SELECT
    session_id,
    blocking_session_id,
    wait_type,
    wait_time,
    status
FROM sys.dm_exec_requests
WHERE blocking_session_id <> 0;
```

Important column:

EXAMPLE / FLOW

```
blocking_session_id
```

Microsoft specifically recommends `sys.dm_exec_requests` and `blocking_session_id` when diagnosing active blocking.

View Locks

SQL

```
SELECT
    request_session_id,
    resource_type,
    request_mode,
    request_status,
    resource_database_id
FROM sys.dm_tran_locks;
```

This can show locks such as:

EXAMPLE / FLOW

```
S
U
X
IS
IX
```

How to Reduce Blocking

Common strategies:

EXAMPLE / FLOW

```
Keep transactions short

Commit or rollback promptly
```

```
Use appropriate indexes

Avoid processing unnecessary rows

Avoid waiting for user input
inside transactions

Use appropriate isolation levels

Consider row-versioning isolation
when appropriate
```

Microsoft emphasizes short, lightweight transactions and good query/application design when reducing blocking.

Deadlock

Simple Definition

A **deadlock** occurs when two or more transactions are waiting on each other in a cycle and none of them can continue.

Blocking:

EXAMPLE / FLOW

```
A blocks B
```

Deadlock:

EXAMPLE / FLOW

```
A waits for B
AND
B waits for A
```

Fundraising Deadlock Example

Session A:

EXAMPLE / FLOW

```
1. Locks Gift 10000
2. Needs Fund 300
```

Session B:

EXAMPLE / FLOW

```
1. Locks Fund 300
2. Needs Gift 10000
```

Diagram:

EXAMPLE / FLOW

```
Session A
   |
   | owns Gift
   v
Gift 10000
   ^
   | wants
Session B


Session B
   |
   | owns Fund
   v
Fund 300
   ^
   | wants
Session A
```

Circular dependency:

EXAMPLE / FLOW

```
A waits for B
B waits for A
```

Neither can continue.

What Does SQL Server Do?

SQL Server detects the deadlock.

It chooses one transaction as the:

EXAMPLE / FLOW

```
deadlock victim
```

That transaction is rolled back.

The other transaction is allowed to continue.

Blocking vs Deadlock

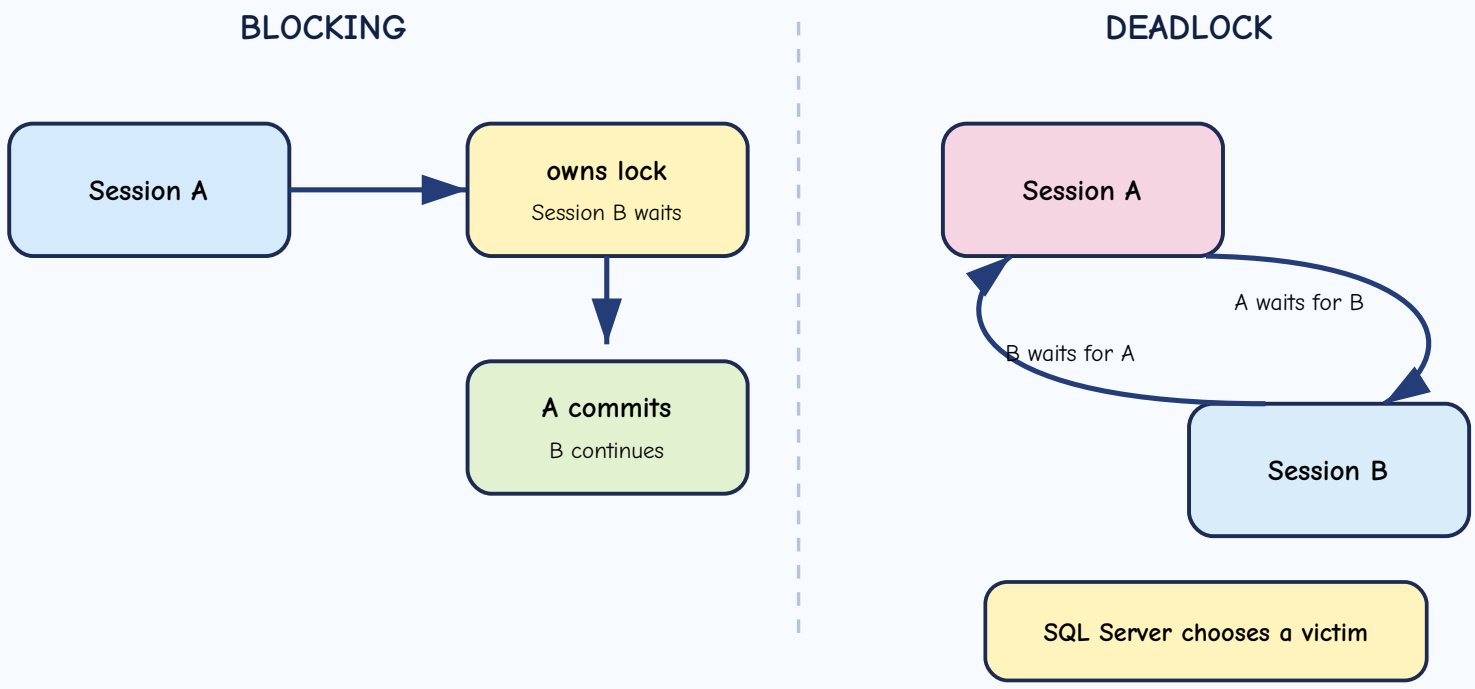
Blocking	Deadlock
One waits for another	Circular waiting
Usually resolves when blocker finishes	Cannot resolve naturally
Normal in small amounts	SQL Server must break cycle

Blocking	Deadlock
No transaction necessarily fails	One transaction is chosen as victim

Easy memory:

FLOW DIAGRAM

Blocking Is a Line - Deadlock Is a Cycle



Blocking normally resolves when the blocker finishes. A deadlock is circular and SQL Server must break the cycle by rolling back one victim.

EXAMPLE / FLOW

```
Blocking
→ line

Deadlock
→ circle
```

Preventing Deadlocks

Keep Transactions Short

Bad:

EXAMPLE / FLOW

```
BEGIN TRAN

Update Gift

Call external API

Wait
```

```
Update Fund

COMMIT
```

Better:

EXAMPLE / FLOW

```
Do external work first

BEGIN TRAN

Perform required DB changes

COMMIT
```

Access Resources in Consistent Order

Bad:

EXAMPLE / FLOW

```
Transaction A:
Gift → Fund

Transaction B:
Fund → Gift
```

Better:

EXAMPLE / FLOW

```
Transaction A:
Gift → Fund

Transaction B:
Gift → Fund
```

Consistent access ordering reduces circular waits. Microsoft recommends consistent object access order and short transactions for deadlock reduction.

Use Good Indexes

A poor query may lock:

EXAMPLE / FLOW

```
100,000 rows
```

when a selective index might let it touch:

EXAMPLE / FLOW

```
10 rows.
```

Smaller lock footprints generally reduce contention.

Use Appropriate Isolation

Higher locking isolation can hold locks longer.

Do not automatically use:

EXAMPLE / FLOW

```
SERIALIZABLE
```

for every operation.

Application Retry

Deadlocks can still occur in correctly designed high-concurrency systems.

Applications should often treat:

EXAMPLE / FLOW

```
deadlock victim
```

as a transient failure that may be safely retried when the business operation is retryable.

Lock Escalation

Simple Definition

Lock escalation converts many small locks into fewer larger locks.

Example:

EXAMPLE / FLOW

```
5,000+ row/key locks
       |
       v
SQL Server may try
lock escalation
       |
       v
Table-level lock
```

The purpose is:

EXAMPLE / FLOW

```
reduce lock-management memory and overhead
```

but the trade-off is:

EXAMPLE / FLOW

```
less concurrency.
```

Microsoft defines lock escalation as converting many fine-grained locks into fewer coarse-grained locks.

Example

Suppose:

SQL

```
UPDATE fundraising.Gift
SET GiftType = 'Adjusted'
WHERE CampaignId = 200;
```

and this affects:

EXAMPLE / FLOW

```
100,000 rows.
```

SQL Server may initially have many lower-level locks.

Conceptually:

EXAMPLE / FLOW

```
Row lock
Row lock
Row lock
Row lock
...
Thousands
       |
       v
Lock Escalation
       |
       v
Larger table-level lock
```

Lock Escalation Threshold

Under normal locking behavior, SQL Server can attempt lock escalation after a statement acquires at least about:

EXAMPLE / FLOW

```
5,000 locks
```

on a single table/index reference, and escalation can also occur due to lock-memory pressure. SQL Server retries escalation checks as more locks are accumulated.

Important:

> ! Do not think that "5,000 locks always means immediate table lock."

It is a trigger/attempt condition, not an absolute guarantee that escalation succeeds.

Preventing Excessive Escalation

Better:

EXAMPLE / FLOW

```
Process huge operations in manageable batches

Keep transactions short

Make queries selective

Use useful indexes
```

Example batch idea:

SQL

```
DELETE TOP (1000)
FROM engagement.Task
WHERE CompletedAtUtc < '2020-01-01';
```

rather than one enormous deletion, when the business operation does not require all rows to be deleted atomically.

Microsoft recommends smaller batches and short transactions rather than disabling lock escalation globally.

Pessimistic Concurrency

Simple Definition

Pessimistic concurrency assumes another transaction may conflict, so it protects data with locks before allowing the operation to continue.

Mental model:

EXAMPLE / FLOW

```
"Someone might change this data.

Lock it now so they cannot interfere."
```

Example

Transaction A:

SQL

```
BEGIN TRANSACTION;
```

```
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift WITH (UPDLOCK)
WHERE GiftId = 10000;

-- calculate something

UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;

COMMIT;
```

The design intentionally protects the row before the final update.

Pessimistic Flow

EXAMPLE / FLOW

```
Read row
   |
   v
Take lock
   |
   v
Other conflicting transaction waits
   |
   v
Update
   |
   v
Commit
   |
   v
Release protection
```

Advantages

EXAMPLE / FLOW

```
Conflicts are prevented early

Useful when conflicts are common

Strong protection during transaction
```

Disadvantages

EXAMPLE / FLOW

```
More blocking

Potential deadlocks

Long transactions hurt concurrency

Locks consume resources
```

When to Use

Consider pessimistic concurrency when:

EXAMPLE / FLOW

```
Conflicting updates are likely

The transaction is short

The application must strongly reserve data
while deciding/updating
```

When Not to Use

Avoid unnecessary pessimistic locking when:

EXAMPLE / FLOW

```
Conflicts are rare

Transactions are long

Many users read the same data

Blocking would seriously reduce throughput
```

Optimistic Concurrency

Simple Definition

Optimistic concurrency assumes conflicts are relatively rare, so transactions proceed without aggressively locking readers and detect conflicts when updates occur.

Mental model:

EXAMPLE / FLOW

```
"Let everyone work.

If somebody changed the same data,
detect the conflict and handle it."
```

Common Application Pattern

Suppose the application reads:

EXAMPLE / FLOW

```
GiftId = 10000
TotalAmount = 1000
RowVersion = 0x0000000000001234
```

Another user updates the Gift.

Its row version changes.

Your application later runs:

SQL

```
UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000
  AND RowVersion = @OriginalRowVersion;
```

If someone already modified the row:

EXAMPLE / FLOW

```
0 rows updated
```

Application detects:

EXAMPLE / FLOW

```
Concurrency conflict
```

and can:

EXAMPLE / FLOW

```
Reload

Tell user

Merge

Retry
```

This is an application-level optimistic-concurrency pattern.

Snapshot Isolation Is Also Optimistic

SQL Server `SNAPSHOT` isolation uses row versions instead of shared read locks.

If a snapshot transaction tries to update a row that another transaction changed after the snapshot transaction began, SQL Server can detect an update conflict and roll the snapshot transaction back.

Optimistic Flow

EXAMPLE / FLOW

```
Transaction A reads version 1

Transaction B reads version 1

Transaction A updates
→ version 2
→ commits

Transaction B tries update
based on old version
        |
        v
Conflict detected
        |
        v
Retry / handle conflict
```

Optimistic vs Pessimistic

Pessimistic	Optimistic
Prevent conflict with locks	Detect conflict later
More blocking	Less reader/writer blocking
Good when conflicts likely	Good when conflicts less frequent
Lock-based protection	Version/conflict-based approach
Can increase deadlocks	Can require retries

What Is Transaction Isolation?

Simple Definition

An **isolation level controls how much one transaction is isolated from changes made by other concurrent transactions.**

It answers questions like:

EXAMPLE / FLOW

```
Can I read uncommitted changes?

If I read a row twice,
must it remain the same?

Can new matching rows appear?
```

```
Should reads block writers?

Should reads use row versions?
```

Main SQL Server Isolation Levels

EXAMPLE / FLOW

```
READ UNCOMMITTED

READ COMMITTED

REPEATABLE READ

SNAPSHOT

SERIALIZABLE
```

SQL Server also supports:

EXAMPLE / FLOW

```
READ COMMITTED SNAPSHOT
```

which is not a separate `SET TRANSACTION ISOLATION LEVEL` keyword; it changes how `READ COMMITTED` works at the database level.

Before Isolation Levels: Three Important Read Problems

You must understand:

EXAMPLE / FLOW

```
Dirty Read

Nonrepeatable Read

Phantom Read
```

Dirty Read

Simple Definition

A **dirty read** occurs when one transaction reads data another transaction changed but has **not committed yet**.

Example

Session A:

SQL

```
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 50000
WHERE GiftId = 10000;
```

Original value:

EXAMPLE / FLOW

```
1000
```

New uncommitted value:

EXAMPLE / FLOW

```
50000
```

Session B reads:

EXAMPLE / FLOW

```
50000
```

Then Session A runs:

SQL

```
ROLLBACK;
```

Final real value:

EXAMPLE / FLOW

```
1000
```

Session B read:

EXAMPLE / FLOW

```
50000
```

which never became permanent.

That is:

EXAMPLE / FLOW

```
Dirty Read
```

Nonrepeatable Read

Simple Definition

A **nonrepeatable read** occurs when the same transaction reads the same row twice and gets different committed values because another transaction changed the row between those reads.

Example

Transaction A:

EXAMPLE / FLOW

```
Read 1:
Gift 10000 = $1000
```

Transaction B:

EXAMPLE / FLOW

```
Update Gift 10000 = $1200
COMMIT
```

Transaction A:

EXAMPLE / FLOW

```
Read 2:
Gift 10000 = $1200
```

Same transaction:

EXAMPLE / FLOW

```
same row
different values
```

That is a nonrepeatable read.

Phantom Read

Simple Definition

A **phantom read** occurs when a transaction repeats a range query and finds new or missing rows because another transaction inserted or deleted rows that match the search condition.

Example

Transaction A:

SQL

```
SELECT COUNT(*)
FROM fundraising.Gift
WHERE CampaignId = 200;
```

Result:

EXAMPLE / FLOW

```
100 Gifts
```

Transaction B:

```sql
INSERT INTO fundraising.Gift (...)
VALUES (... CampaignId = 200 ...);

COMMIT;
```

Transaction A repeats:

```sql
SELECT COUNT(*)
FROM fundraising.Gift
WHERE CampaignId = 200;
```

Result:

```
101 Gifts
```

The extra matching row is the:

```
phantom
```

READ UNCOMMITTED

Simple Definition

`READ UNCOMMITTED` provides the weakest traditional isolation.

It allows:

```
Dirty reads

Nonrepeatable reads

Phantom reads
```

Microsoft's isolation-level documentation confirms that all three can occur under `READ UNCOMMITTED`.

Syntax

```sql
SET TRANSACTION ISOLATION LEVEL
READ UNCOMMITTED;
```

Equivalent NOLOCK Concept

You often see:

```sql
SELECT ...
FROM fundraising.Gift WITH (NOLOCK);
```

`NOLOCK` gives `READ UNCOMMITTED`-style behavior for the referenced table.

Example

Session A:

```sql
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 50000
WHERE GiftId = 10000;
```

No commit yet.

Session B:

```sql
SET TRANSACTION ISOLATION LEVEL
READ UNCOMMITTED;

SELECT TotalAmount
FROM fundraising.Gift
WHERE GiftId = 10000;
```

Session B may read:

```
50000
```

even though Session A could later roll back.

Advantage

```
Less read blocking from ordinary data locks
```

Major Disadvantage

The data can be:

EXAMPLE / FLOW

```
Uncommitted

Inconsistent

Changed while being read
```

For financial/fundraising reporting:

EXAMPLE / FLOW

```
incorrect totals
```

can be unacceptable.

When to Use

Rarely for business-critical data.

Potential specialized use:

EXAMPLE / FLOW

```
Approximate diagnostic/reporting query
where temporary inconsistency is explicitly acceptable
```

When Not to Use

Avoid for:

EXAMPLE / FLOW

```
Financial totals

Donation processing

Critical reports

Anything that requires trustworthy committed data
```

Important Rule

Do not use:

EXAMPLE / FLOW

```
NOLOCK everywhere
```

as a generic fix for blocking.

It trades blocking for weaker correctness.

READ COMMITTED

Simple Definition

`READ COMMITTED` prevents dirty reads.

A transaction cannot read another transaction's uncommitted changes.

It is the default isolation level in SQL Server.

Syntax

SQL

```
SET TRANSACTION ISOLATION LEVEL
READ COMMITTED;
```

What It Prevents

EXAMPLE / FLOW

```
Dirty Read
→ Prevented
```

But allows:

EXAMPLE / FLOW

```
Nonrepeatable Read
→ Possible

Phantom Read
→ Possible
```

Lock-Based READ COMMITTED

On SQL Server, when:

EXAMPLE / FLOW

```
READ_COMMITTED_SNAPSHOT = OFF
```

which is the default for SQL Server databases, `READ COMMITTED` uses shared locking for reads.

Conceptually:

EXAMPLE / FLOW

```
SELECT reads committed data
      |
      v
Shared lock while reading
      |
      v
Do not read uncommitted writer's row
```

Example

Transaction A:

SQL

```
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;
```

No commit.

Transaction B:

SQL

```
SELECT TotalAmount
FROM fundraising.Gift
WHERE GiftId = 10000;
```

Under lock-based `READ COMMITTED`, Transaction B may wait until A:

EXAMPLE / FLOW

```
COMMITs
or
ROLLBACKs
```

because it cannot read the uncommitted version.

Why Can Values Change Between Statements?

Shared read locks at ordinary lock-based `READ COMMITTED` are generally not held until the end of the complete transaction.

Therefore:

EXAMPLE / FLOW

```
Transaction A:
Read Gift = 1000
```

```
Transaction B:
Update Gift = 1200
COMMIT

Transaction A:
Read Gift again = 1200
```

is possible.

That is:

EXAMPLE / FLOW

```
Nonrepeatable Read
```

REPEATABLE READ

Simple Definition

`REPEATABLE READ` prevents another transaction from modifying rows that your transaction has already read until your transaction completes.

It prevents:

EXAMPLE / FLOW

```
Dirty reads

Nonrepeatable reads
```

but still allows:

EXAMPLE / FLOW

```
Phantom reads
```

Syntax

SQL

```
SET TRANSACTION ISOLATION LEVEL
REPEATABLE READ;
```

Example

Transaction A:

```sql
SET TRANSACTION ISOLATION LEVEL
REPEATABLE READ;

BEGIN TRANSACTION;

SELECT
    TotalAmount
FROM fundraising.Gift
WHERE GiftId = 10000;
```

Transaction A keeps stronger read protection until transaction completion.

Transaction B attempts:

```sql
UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;
```

Transaction B may wait.

Transaction A reads again:

```sql
SELECT
    TotalAmount
FROM fundraising.Gift
WHERE GiftId = 10000;
```

The value remains repeatable during A's transaction.

Why Are Phantoms Still Possible?

Suppose A runs:

```sql
SELECT *
FROM fundraising.Gift
WHERE CampaignId = 200;
```

Existing matching rows are protected.

But another transaction may still insert a completely new row:

```
CampaignId = 200
```

depending on the range involved.

Therefore when A repeats the query:

```
new matching row may appear
```

That is a phantom.

Advantage

Useful when:

EXAMPLE / FLOW

```
rows already read must not change
during the transaction.
```

Disadvantage

Shared locks are held longer.

Result:

EXAMPLE / FLOW

```
More blocking

Lower concurrency

Potentially more deadlock opportunity
```

SERIALIZABLE

Simple Definition

`SERIALIZABLE` is the strongest traditional lock-based isolation level.

It tries to make concurrent transactions behave as though they had executed one after another rather than at the same time.

It prevents:

EXAMPLE / FLOW

```
Dirty reads

Nonrepeatable reads

Phantom reads
```

Syntax

SQL

```
SET TRANSACTION ISOLATION LEVEL
SERIALIZABLE;
```

How Does It Prevent Phantoms?

SQL Server can use:

EXAMPLE / FLOW

```
key-range locks
```

so another transaction cannot insert a new row into a protected search range until the transaction completes.

Example

Transaction A:

SQL

```sql
SET TRANSACTION ISOLATION LEVEL
SERIALIZABLE;

BEGIN TRANSACTION;

SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE CampaignId = 200;
```

Transaction B tries to insert a new Gift that belongs to Campaign 200.

Depending on the indexes and range being protected:

EXAMPLE / FLOW

```
Transaction B may have to wait.
```

Therefore Transaction A sees a stable set of matching rows.

SERIALIZABLE Diagram

EXAMPLE / FLOW

```
Protected key range

Campaign 200 Gifts
|----------------------------|
Gift A
Gift B
Gift C

No other transaction can
change the protected rows
or insert a new matching row
```

```
into the protected range
until transaction completes.
```

Advantage

Maximum traditional isolation.

Useful for special business rules where:

EXAMPLE / FLOW

```
the complete set of matching rows
must remain stable.
```

Disadvantages

EXAMPLE / FLOW

```
More locking

More blocking

Lower concurrency

Higher deadlock potential
```

Do not use `SERIALIZABLE` everywhere.

SNAPSHOT Isolation

Simple Definition

`SNAPSHOT` isolation uses **row versions** rather than shared read locks to provide a consistent view of data for the entire transaction.

Think:

> ! "Show me the committed data as it looked when my transaction's snapshot began."

SQL Server's snapshot isolation provides transaction-level read consistency using row versioning.

Enable SNAPSHOT Support

Database option:

SQL

```
ALTER DATABASE FundraisingDb
SET ALLOW_SNAPSHOT_ISOLATION ON;
```

Then a session can use:

```sql
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
```

SNAPSHOT Example

Initial Gift:

```
Gift 10000 = $1000
```

Transaction A:

```sql
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;

BEGIN TRANSACTION;

SELECT TotalAmount
FROM fundraising.Gift
WHERE GiftId = 10000;
```

Returns:

```
1000
```

Transaction B:

```sql
UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;

COMMIT;
```

Transaction A runs the SELECT again.

Under **SNAPSHOT**, it still sees:

```
1000
```

because its transaction-level snapshot remains consistent.

SNAPSHOT Timeline

EXAMPLE / FLOW

```
Time ------------------------------------------------>

Transaction A starts
Snapshot sees Gift = 1000
      |
      |--------------------------|
                                 |
Transaction B updates            |
Gift = 1200                      |
COMMIT                           |
                                 |
Transaction A reads again -------+
Still sees 1000
```

Does SNAPSHOT Mean No Locks at All?

No.

Snapshot reads avoid normal shared row/page locks, but SQL Server still uses other locks such as schema stability locks, and writers still need write protection.

Microsoft explicitly states that snapshot reads do not acquire page/row locks for normal data reads, while modifications continue to require concurrency control.

Update Conflict Under SNAPSHOT

Suppose:

EXAMPLE / FLOW

```
Transaction A starts
and reads Gift = 1000

Transaction B updates Gift = 1200
and commits

Transaction A then tries
to update the same row
```

SQL Server may detect:

EXAMPLE / FLOW

```
the row changed since A's snapshot
```

and reject A's update.

This is optimistic conflict detection.

Advantages

EXAMPLE / FLOW

```
Readers generally do not block writers

Writers generally do not block snapshot readers

Stable transaction-level view

Can reduce read/write deadlocks
```

Disadvantages

EXAMPLE / FLOW

```
Row-version storage overhead

Additional database/tempdb/PVS resources
depending on configuration

Update conflicts can occur

Applications may need retry logic
```

READ COMMITTED SNAPSHOT - RCSI

This is one of the most important modern SQL Server concurrency features.

Simple Definition

Read Committed Snapshot Isolation (RCSI) changes `READ COMMITTED` so reads use row versions instead of ordinary shared row/page locks.

The isolation semantics remain:

EXAMPLE / FLOW

```
READ COMMITTED
```

but the implementation changes from:

EXAMPLE / FLOW

```
locking reads
```

to:

EXAMPLE / FLOW

```
row-versioned reads.
```

Enable RCSI

SQL

```
ALTER DATABASE FundraisingDb
SET READ_COMMITTED_SNAPSHOT ON;
```

After this, sessions using normal:

EXAMPLE / FLOW

```
READ COMMITTED
```

automatically use row-versioned reads.

You do **not** write:

SQL

```
SET TRANSACTION ISOLATION LEVEL
READ COMMITTED SNAPSHOT;
```

There is no such isolation-level syntax.

RCSI is a database option controlling the implementation of `READ COMMITTED`.

RCSI Behavior

Suppose Transaction A:

SQL

```
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;
```

has not committed.

Transaction B:

SQL

```
SELECT TotalAmount
FROM fundraising.Gift
WHERE GiftId = 10000;
```

With lock-based `READ COMMITTED`:

EXAMPLE / FLOW

```
B may wait.
```

With RCSI:

EXAMPLE / FLOW

```
B reads the last committed row version
instead of waiting on the uncommitted update.
```

RCSI Flow

EXAMPLE / FLOW

```
Current row
Gift = 1200
(uncommitted by writer)
       |
       +---- row version chain
                    |
                    v
             previous committed
             Gift = 1000
                    |
                    v
RCSI reader sees 1000
```

RCSI Consistency Level

This is critical.

RCSI gives:

EXAMPLE / FLOW

```
statement-level consistency
```

Each statement sees committed data as of the beginning of that statement.

Therefore:

EXAMPLE / FLOW

```
SELECT 1
```

and:

EXAMPLE / FLOW

```
SELECT 2
```

within the same transaction can see different committed database states if another transaction commits between the statements.

RCSI Example

Transaction A:

EXAMPLE / FLOW

```
Statement 1
Gift = 1000
```

Transaction B:

EXAMPLE / FLOW

```
updates Gift = 1200
COMMIT
```

Transaction A:

EXAMPLE / FLOW

```
Statement 2
Gift = 1200
```

This is valid under RCSI because each statement gets its own committed snapshot.

RCSI vs SNAPSHOT

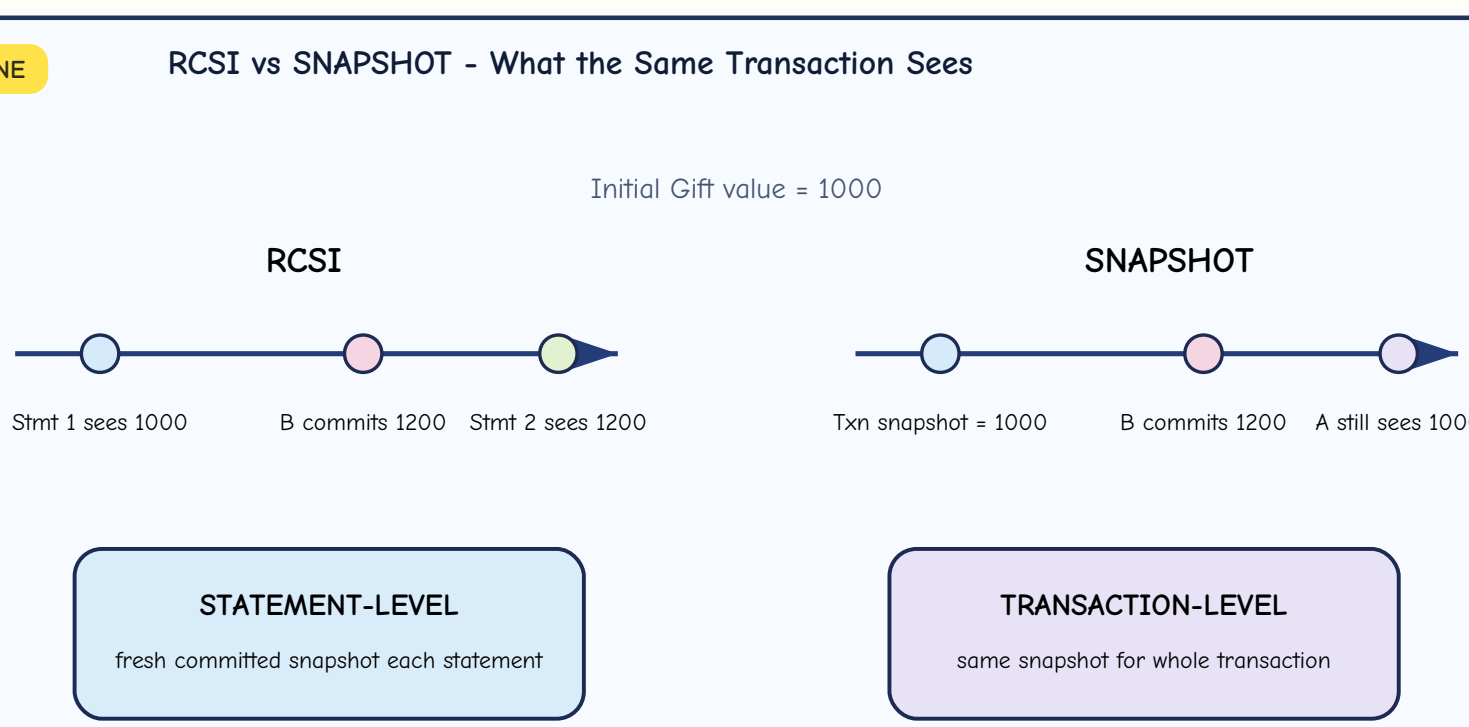
This distinction is extremely important.

EXAMPLE / FLOW

```
RCSI
→ Snapshot per statement

SNAPSHOT
→ Snapshot per transaction
```

TIMELINE



RCSI can observe a newly committed value on the next statement; SNAPSHOT keeps the transaction's original row-versioned view.

Example

Initial:

EXAMPLE / FLOW

```
Gift = 1000
```

Transaction A begins.

First query:

EXAMPLE / FLOW

```
1000
```

Transaction B updates:

EXAMPLE / FLOW

```
1200
```

and commits.

Transaction A queries again.

Under RCSI:

EXAMPLE / FLOW

```
1200
```

because the second statement begins after B committed.

Under SNAPSHOT:

EXAMPLE / FLOW

```
1000
```

because A keeps the same transaction-level snapshot.

RCSI vs SNAPSHOT Table

RCSI	SNAPSHOT
Database option `READ_COMMITTED_SNAPSHOT ON`	Database option `ALLOW_SNAPSHOT_ISOLATION ON`
Uses `READ COMMITTED` isolation	Session explicitly selects `SNAPSHOT`
Statement-level consistency	Transaction-level consistency
New snapshot each statement	Same snapshot during transaction
Reduces read/write blocking	Reduces read/write blocking
No explicit isolation change needed by application	Application/session must request SNAPSHOT

Important SQL Server Defaults

For SQL Server:

EXAMPLE / FLOW

```
READ COMMITTED
→ default isolation level
```

and for normal SQL Server databases:

EXAMPLE / FLOW

```
READ_COMMITTED_SNAPSHOT
→ OFF by default
```

so traditional `READ COMMITTED` normally uses shared locks for reads unless the database option is changed.

Azure SQL Database differs because RCSI is commonly enabled by default.

Isolation-Level Comparison

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SNAPSHOT	Prevented	Prevented	Prevented
SERIALIZABLE	Prevented	Prevented	Prevented

Microsoft documents these core concurrency effects for the isolation levels.

Isolation Strength Diagram

Traditional locking levels:

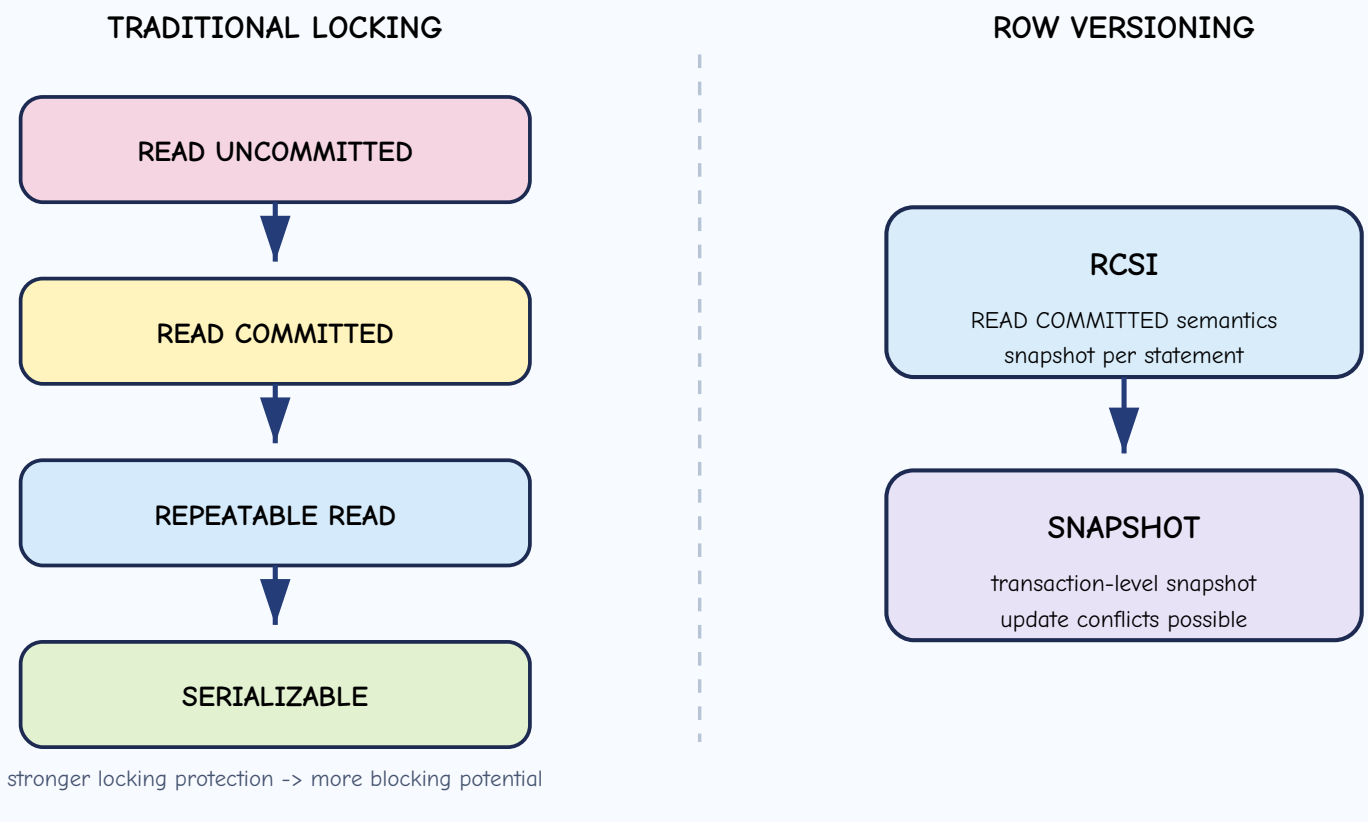
EXAMPLE / FLOW

```
Less Isolation
      |
      v
READ UNCOMMITTED
      |
      v
READ COMMITTED
      |
      v
```

```
REPEATABLE READ
      |
      v
SERIALIZABLE
      |
      v
More Isolation
```

ISOLATION MAP

Traditional Locking Levels and Row-Versioned Alternatives



RCSI and SNAPSHOT reduce ordinary reader/writer blocking by using row versions, but their consistency scope is different.

But:

EXAMPLE / FLOW

SNAPSHOT

is different because it provides high read consistency using row versions rather than simply taking progressively stronger shared locks.

Blocking Behavior Comparison

READ UNCOMMITTED

EXAMPLE / FLOW

```
Readers
→ minimal normal read locking

Risk
→ dirty/inconsistent reads
```

READ COMMITTED - locking version

EXAMPLE / FLOW

Readers use shared locks

Writer can block reader

Reader can temporarily block writer

REPEATABLE READ

EXAMPLE / FLOW

Shared locks held longer

More stable values

More blocking

SERIALIZABLE

EXAMPLE / FLOW

Locks rows + ranges

Strongest traditional protection

Highest blocking potential

RCSI

EXAMPLE / FLOW

Readers use row versions

Read/write blocking reduced

Statement-level snapshot

SNAPSHOT

EXAMPLE / FLOW

Readers use row versions

Transaction-level snapshot

Update conflicts possible

Which Isolation Level Should You Use?

Do not automatically choose:

EXAMPLE / FLOW

```
highest isolation
```

because stronger protection can reduce concurrency.

Choose according to the business requirement.

READ UNCOMMITTED

Use only when:

EXAMPLE / FLOW

```
Inconsistent values are acceptable
```

Rare for important fundraising data.

READ COMMITTED

Good general starting point.

It prevents dirty reads while preserving reasonable concurrency.

RCSI

A strong option for OLTP systems with significant:

EXAMPLE / FLOW

```
read/write blocking
```

because it preserves `READ COMMITTED` semantics while reducing shared-lock read blocking.

Microsoft's current deadlock guidance specifically recommends row-versioning-based `READ COMMITTED` for applications unless the application depends on lock-based read blocking behavior.

But enabling it is still a database-level design decision that must be tested.

REPEATABLE READ

Use when:

EXAMPLE / FLOW

```
Rows already read must remain unchanged
until transaction completion
```

but new matching rows do not necessarily need to be prevented.

SNAPSHOT

Use when:

EXAMPLE / FLOW

```
The entire transaction should read
a consistent historical snapshot

and optimistic update-conflict handling
is acceptable.
```

SERIALIZABLE

Use when:

EXAMPLE / FLOW

```
Rows and qualifying ranges
must remain stable
```

and the stronger blocking/concurrency cost is justified.

Practical Fundraising Scenario

Suppose Sarah is reviewing:

EXAMPLE / FLOW

```
John Smith's Gift history
```

while another process records a new Gift.

READ COMMITTED

Sarah's first query:

EXAMPLE / FLOW

```
5 Gifts
```

New Gift commits.

Sarah's second query:

EXAMPLE / FLOW

```
6 Gifts
```

Possible.

REPEATABLE READ

Existing rows Sarah read are protected from modification.

But a new Gift can potentially appear:

EXAMPLE / FLOW

```
5
then
6
```

because phantom inserts are still possible.

SERIALIZABLE

SQL Server can protect the matching key range.

Sarah may continue seeing:

EXAMPLE / FLOW

```
5 Gifts
```

and another transaction trying to insert a matching Gift may wait.

RCSI

Sarah's first statement:

EXAMPLE / FLOW

```
5 Gifts
```

New Gift commits.

Sarah's second statement:

EXAMPLE / FLOW

```
6 Gifts
```

because each statement gets a fresh committed snapshot.

SNAPSHOT

Sarah's transaction starts with:

EXAMPLE / FLOW

```
5 Gifts
```

New Gift commits elsewhere.

Sarah continues seeing:

EXAMPLE / FLOW

```
5 Gifts
```

until her transaction ends.

Checking RCSI and SNAPSHOT Settings

SQL

```sql
SELECT
    name,

    is_read_committed_snapshot_on,

    snapshot_isolation_state_desc

FROM sys.databases

WHERE name = 'FundraisingDb';
```

Interpretation:

EXAMPLE / FLOW

```
is_read_committed_snapshot_on = 1
→ RCSI enabled

snapshot_isolation_state_desc = ON
→ SNAPSHOT transactions allowed
```

Microsoft exposes both settings through `sys.databases`.

Enabling RCSI

SQL

```sql
ALTER DATABASE FundraisingDb
SET READ_COMMITTED_SNAPSHOT ON;
```

Important:

Changing this option requires control over active database connections during the change, so it should be planned carefully in a real production environment.

Enabling SNAPSHOT

SQL

```
ALTER DATABASE FundraisingDb
SET ALLOW_SNAPSHOT_ISOLATION ON;
```

Then a session can use:

SQL

```
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
```

Locking + Isolation Mental Model

EXAMPLE / FLOW

```
Isolation requirement
       |
       v
How much protection is needed?
       |
       +---------------------------+
       |                           |
 Lock-based                  Row-version based
       |                           |
       v                           v
READ COMMITTED               RCSI
REPEATABLE READ              SNAPSHOT
SERIALIZABLE
```

Common Concurrency Mistakes

Thinking Blocking Is Always Bad

Some blocking is normal.

The problem is:

EXAMPLE / FLOW

```
long or excessive blocking.
```

Thinking Deadlock and Blocking Are the Same

They are not.

EXAMPLE / FLOW

```
Blocking
→ one waits

Deadlock
→ circular wait
```

Using NOLOCK Everywhere

`NOLOCK` can expose:

EXAMPLE / FLOW

```
dirty reads
```

and inconsistent results.

Do not use it simply because a report is slow.

Holding Transactions Open

Bad:

SQL

```sql
BEGIN TRANSACTION;

UPDATE ...;

-- application waits for user

COMMIT;
```

Keep transactions short.

Using SERIALIZABLE Everywhere

This can cause:

EXAMPLE / FLOW

```
heavy blocking
reduced concurrency
deadlocks
```

Use only when its guarantees are actually required.

Assuming READ COMMITTED Always Uses Shared Read Locks

Not necessarily.

If:

EXAMPLE / FLOW

```
READ_COMMITTED_SNAPSHOT = ON
```

then `READ COMMITTED` uses row-versioned reads instead.

Confusing RCSI and SNAPSHOT

Remember:

EXAMPLE / FLOW

```
RCSI
→ statement snapshot

SNAPSHOT
→ transaction snapshot
```

Thinking SNAPSHOT Means No Locks Anywhere

Writes still require concurrency protection, and schema locks still exist.

Snapshot mainly changes how normal data reads are performed.

Disabling Lock Escalation Immediately

Lock escalation helps SQL Server control lock-memory usage.

First fix:

EXAMPLE / FLOW

```
large transactions

poor queries

unnecessary row counts
```

before attempting to disable escalation.

Processing Huge Changes in One Transaction

If the business operation does not require one huge atomic transaction, controlled batching may reduce:

EXAMPLE / FLOW

```
lock count

blocking

log usage

escalation risk
```

Concurrency Best Practices

Use these rules:

EXAMPLE / FLOW

```
Keep transactions short
```

```
Commit/rollback quickly

Access objects in consistent order

Index important search/join columns

Avoid touching unnecessary rows

Use the lowest isolation level
that still satisfies correctness

Consider RCSI for read/write blocking
after proper testing

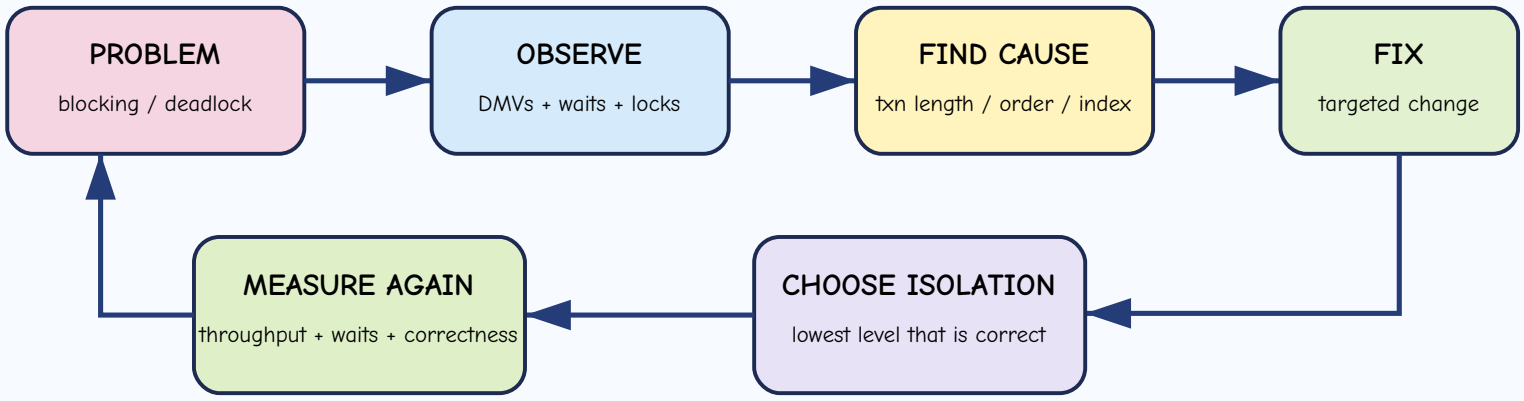
Handle deadlocks with retry logic
when appropriate

Use optimistic concurrency tokens
for disconnected application updates

Measure blocking rather than guessing
```

FINAL FLOW

Concurrency Troubleshooting and Design Loop



Keep transactions short, access resources consistently, index selective work, and test row-versioning choices.

Concurrency tuning is evidence-driven: identify the wait pattern, fix the real cause, choose the correct isolation behavior, and verify again.

Practical Two-Session Blocking Lab

Open:

EXAMPLE / FLOW

```
SSMS Window 1

SSMS Window 2
```

Window 1

SQL

```sql
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET Notes = 'Testing blocking'
WHERE GiftId = 10000;

-- Do not commit yet.
```

Window 2

SQL

```sql
UPDATE fundraising.Gift
SET Notes = 'Window 2'
WHERE GiftId = 10000;
```

Window 2 should wait if Window 1 still owns conflicting protection.

Window 1

SQL

```sql
ROLLBACK;
```

Now Window 2 can continue.

This demonstrates:

EXAMPLE / FLOW

```
Exclusive lock
+
Blocking
+
Transaction duration
```

Practical Deadlock Pattern

Conceptual sessions:

Session A

```sql
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET Notes = 'A'
WHERE GiftId = 10000;

-- Then access Fund 300
```

Session B

```sql
BEGIN TRANSACTION;

UPDATE fundraising.Fund
SET Description = Description
WHERE FundId = 300;

-- Then access Gift 10000
```

If timing aligns:

```
EXAMPLE / FLOW
A owns Gift
A wants Fund

B owns Fund
B wants Gift
```

Deadlock.

Better design:

```
EXAMPLE / FLOW
Both transactions access:

Gift first
then Fund
```

or both:

```
EXAMPLE / FLOW
Fund first
then Gift
```

The important point is consistency.

Interview Questions

What is concurrency?

Concurrency means multiple database operations execute during overlapping time periods.

SQL Server must preserve data correctness while allowing as much simultaneous work as possible.

What is locking?

Locking is SQL Server's mechanism for controlling concurrent access to database resources.

Locks protect rows, keys, pages, tables, and other resources while transactions read or modify them.

What is a Shared lock?

A Shared lock protects data being read.

Multiple compatible readers can usually hold shared locks at the same time.

What is an Exclusive lock?

An Exclusive lock protects data being modified.

Conflicting transactions must wait until the exclusive protection is released.

What is an Update lock?

An Update lock protects data that may later be updated and helps prevent a common shared-to-exclusive conversion deadlock.

What is an Intent lock?

An intent lock indicates that more detailed locks exist or are intended at a lower level of the lock hierarchy.

Examples:

EXAMPLE / FLOW

```
IS
IX
IU
```

What is blocking?

Blocking occurs when one session waits because another transaction owns an incompatible lock on a required resource.

Is blocking always bad?

No.

Short blocking is a normal part of concurrency control.

Long or widespread blocking is what usually needs investigation.

What is a deadlock?

A deadlock is a circular dependency where transactions wait for resources held by each other.

SQL Server detects the cycle and rolls back one victim.

Blocking vs deadlock?

EXAMPLE / FLOW

```
Blocking
→ waiting chain

Deadlock
→ waiting cycle
```

What is lock escalation?

Lock escalation converts many fine-grained locks into fewer coarse-grained locks, commonly a table-level lock, to reduce lock-management overhead.

When can lock escalation occur?

One trigger is when a statement acquires around 5,000 locks on a single table/index reference; lock-memory pressure can also trigger escalation attempts.

What is pessimistic concurrency?

Pessimistic concurrency prevents conflicts in advance by acquiring locks.

Other conflicting operations wait.

What is optimistic concurrency?

Optimistic concurrency allows concurrent work and detects conflicting changes later.

The application may then retry or resolve the conflict.

What is an isolation level?

An isolation level controls what one transaction can observe from other concurrent transactions and how much protection is used.

What is a dirty read?

A dirty read is reading another transaction's uncommitted change.

What is a nonrepeatable read?

A nonrepeatable read occurs when the same row is read twice within one transaction and returns different committed values.

What is a phantom read?

A phantom read occurs when repeating a range query returns additional or missing matching rows due to another committed insert/delete.

What does READ UNCOMMITTED allow?

It allows:

EXAMPLE / FLOW

```
Dirty reads

Nonrepeatable reads

Phantom reads
```

What does READ COMMITTED guarantee?

It prevents dirty reads.

It is SQL Server's default isolation level.

What does REPEATABLE READ guarantee?

Rows already read cannot be modified by other transactions until the transaction completes.

It prevents dirty and nonrepeatable reads but can still allow phantoms.

What does SERIALIZABLE guarantee?

It prevents:

EXAMPLE / FLOW

```
Dirty reads

Nonrepeatable reads

Phantom reads
```

and can protect key ranges against new matching inserts.

What is SNAPSHOT isolation?

SNAPSHOT uses row versions to provide a transaction-level consistent view of data.

Readers usually do not block writers, but update conflicts can cause a snapshot transaction to fail.

What is RCSI?

RCSI stands for:

EXAMPLE / FLOW

```
READ COMMITTED SNAPSHOT
```

It changes the normal `READ COMMITTED` implementation so reads use row versions rather than shared row/page locks.

It provides:

EXAMPLE / FLOW
```
statement-level consistency.
```

RCSI vs SNAPSHOT?

The easiest interview answer is:

EXAMPLE / FLOW
```
RCSI
→ READ COMMITTED semantics
→ snapshot per statement

SNAPSHOT
→ explicit SNAPSHOT isolation
→ snapshot for entire transaction
```

RCSI uses:

SQL
```
ALTER DATABASE ...
SET READ_COMMITTED_SNAPSHOT ON;
```

SNAPSHOT uses:

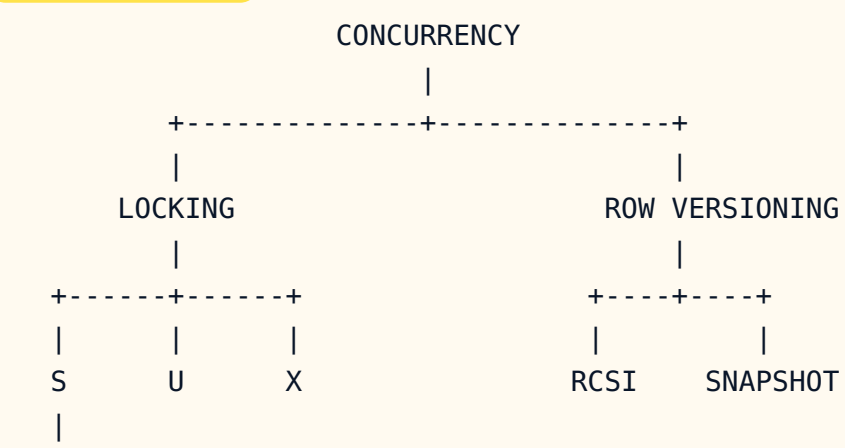
SQL
```
ALTER DATABASE ...
SET ALLOW_SNAPSHOT_ISOLATION ON;
```

plus:

SQL
```
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
```

Final Master Diagram

EXAMPLE / FLOW



```
+---- Intent Locks
|       IS / IX / IU
|
+---- Blocking
|
+---- Deadlocks
|
+---- Lock Escalation
```

Isolation:

EXAMPLE / FLOW

```
READ UNCOMMITTED
      |
      v
READ COMMITTED
      |
      v
REPEATABLE READ
      |
      v
SERIALIZABLE
```

Version-based alternatives:

EXAMPLE / FLOW

```
READ COMMITTED
   +
RCSI ON
   |
   v
Statement-level snapshots
```

and:

EXAMPLE / FLOW

```
SNAPSHOT
   |
   v
Transaction-level snapshot
```

Final Easy Memory Rules

EXAMPLE / FLOW

```
Concurrency
→ Multiple users working at the same time
```

EXAMPLE / FLOW

Lock
→ Protect database resource

EXAMPLE / FLOW

S Lock
→ Read

EXAMPLE / FLOW

X Lock
→ Write

EXAMPLE / FLOW

U Lock
→ Row may be updated

EXAMPLE / FLOW

Intent Lock
→ Lower-level locks exist/intended

EXAMPLE / FLOW

Blocking
→ One transaction waits

EXAMPLE / FLOW

Deadlock
→ Transactions wait in a circle

EXAMPLE / FLOW

Lock Escalation
→ Many small locks become fewer large locks

EXAMPLE / FLOW

Pessimistic Concurrency
→ Prevent conflict first

EXAMPLE / FLOW

Optimistic Concurrency
→ Detect conflict later

EXAMPLE / FLOW

Dirty Read
→ Read uncommitted data

EXAMPLE / FLOW

Nonrepeatable Read
→ Same row changes between reads

EXAMPLE / FLOW

Phantom Read
→ New/missing matching rows appear

EXAMPLE / FLOW

READ UNCOMMITTED
→ Dirty reads allowed

EXAMPLE / FLOW

READ COMMITTED
→ No dirty reads

EXAMPLE / FLOW

REPEATABLE READ
→ Previously read rows cannot change

EXAMPLE / FLOW

SERIALIZABLE
→ Rows and ranges stay stable

EXAMPLE / FLOW

RCSI
→ READ COMMITTED
using row versions
per statement

EXAMPLE / FLOW

SNAPSHOT
→ One row-versioned view
for the transaction

The most important overall rule is:

! Concurrency is a balance between correctness and simultaneous access. More locking and stronger isolation can protect data more aggressively but may increase blocking and deadlocks. Row-versioning options such as RCSI and SNAPSHOT can reduce reader/writer blocking, but they add version-storage costs and require different conflict-handling behavior. Choose the lowest isolation level that still guarantees the correctness required by the business operation.